

Humans versus AI: A Multi-Dimensional Characterization of Code Quality and Security

TODO Author 

TODO Affiliation, Country

Abstract

AI has officially moved from a pilot technology to a standard component of the software development lifecycle. AI coding assistants are approaching universal adoption among developers and organizations, yet the software engineering community still lacks a unified empirical picture of how AI-generated code compares to human-written code across quality and security dimensions. Prior research provides inconsistent findings because studies rely on different datasets and evaluation toolchains, making it difficult to determine whether observed patterns are broadly valid or specific to a particular experimental setup. To address this, we analyse CodeMirage, a large-scale multilingual benchmark covering human-authored and AI-generated code from multiple production-level LLMs. We evaluate metrics across four categories: stylistic violations, structural complexity, security vulnerability profiles, and maintainability indicators, with all comparisons grounded in nonparametric statistical tests and effect sizes. The results reveal that AI-generated code carries $4.6\times$ higher violation densities overall, though the language's type discipline strongly moderates this gap: statically typed languages narrow it to $1.32\times$ while dynamically typed languages widen it to $2.75\times$. Security vulnerability distributions follow distinct patterns across origin, and when comparing open-weight and closed-weight LLMs on the same benchmark, measurable differences emerge in both quality and security profiles. These findings establish an empirical baseline for evaluating generation models, training developers to supervise AI output across language contexts, and building detection and attribution tools on reproducible evidence.

2012 ACM Subject Classification Software and its engineering $\rightarrow$ Software defect analysis; Software and its engineering $\rightarrow$ Software testing and debugging

Keywords and phrases TODO: comma-separated keywords

Digital Object Identifier 10.4230/LIPIcs.TODO.2025.TODO

Funding *TODO Author*: TODO funding

Acknowledgements TODO: Acknowledgments text. Include funding sources, data providers, or reviewers as appropriate.

1 Introduction

AI-assisted programming tools have transitioned from experimental to standard practice across the software industry. As of 2025, 85% of developers regularly use AI tools for coding and software design, and 51% of professional developers report using them daily [33]. Approximately 46% of newly written code is now AI-assisted, a figure projected to reach 60% by the end of 2026 [33]. GitHub Copilot alone has accumulated 20 million cumulative users and is deployed by 90% of Fortune 100 companies, while the AI coding tools market reached \$7.37 billion in 2025 [33]. Systems such as Copilot, Devin, and Cursor automate tasks ranging from code completion to full pull request authorship, with minimal human oversight at each step.

This adoption is accompanied by documented quality concerns. Developer trust in AI output has declined sharply: only 29% of developers reported confidence in AI-generated code in 2025, down from above 70% in 2023, and 45% say that debugging AI-generated code takes more time than writing equivalent code manually [33]. Industry analyses report



© TODO Author Names;
licensed under Creative Commons License CC-BY 4.0
TODO Conference Name (TODO 2025).

Editors: TODO; Article No. TODO; pp. TODO:1–TODO:17



Leibniz International Proceedings in Informatics
LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

2.74 times more vulnerabilities in AI-generated code relative to human-authored code, with 45% of AI code samples failing security tests outright [33]. Prior research confirms that AI-generated code exhibits higher defect rates, yet leaves the broader picture incomplete in four ways. First, most studies are restricted to Python or Python and Java, leaving cross-language patterns and the moderating role of type discipline untested at scale. Second, each study typically examines a single quality dimension – stylistic violations, defect rates, or security findings – rather than applying a unified framework. Third, statistical approaches vary from descriptive-only reporting to parametric tests on non-normal distributions, with effect sizes absent throughout. Fourth, no existing work evaluates multiple production-level LLMs jointly on the same benchmark, making it unclear whether observed patterns reflect a particular model or AI-generated code more broadly.

We address this through a large-scale empirical analysis of the CodeMirage benchmark [15], a multilingual dataset comprising approximately 110,000 files spanning 10 programming languages and generated by 10 production-level LLMs alongside a human-authored baseline. The LLM pool includes both closed-weight models (Claude-3.5-Haiku, GPT-4o-mini, GPT-o3-mini, Gemini-2.0-Flash, Gemini-2.0-Flash-Thinking, Gemini-2.0-Pro) and open-weight models (DeepSeek-V3, DeepSeek-R1, Llama-3.3-70B, Qwen-2.5-Coder-32B), enabling a direct comparison of model families alongside the AI-versus-human analysis. We apply language-specific linters, structural complexity measurement, and security scanning uniformly across all languages, with all comparisons grounded in nonparametric statistical tests and effect sizes, stratified by type discipline to examine whether static type systems moderate AI-associated defect risk. This study examines three research questions:

RQ1: *What structural and stylistic quality differences exist between AI-generated and human-written code?*

RQ2: *How do security vulnerability profiles differ between AI-generated and human-written code?*

RQ3: *Do open-weight LLMs produce code with measurably different quality and security profiles compared to closed-weight LLMs when evaluated on the same benchmark dataset?*

The main contributions of this work are as follows:

1. **Large-scale multi-language empirical comparison across 10 languages and 10 LLMs.** We analyse the CodeMirage benchmark using a unified framework covering stylistic, structural, and security quality dimensions, providing systematic evidence of how AI authorship and model family affect code quality across language families.
2. **Evidence that type discipline moderates AI-associated quality gaps.** Statically-typed languages (Java, C#, TypeScript) exhibit a mean 1.66-fold violation density gap between AI-generated and human-authored code, compared to a 2.34-fold gap in dynamically-typed languages. A chi-square test confirms this pattern ($p < 0.001$), suggesting that compiler-enforced type checking intercepts a class of errors that otherwise persist in dynamic languages.
3. **Characterisation of AI-specific security vulnerability patterns.** AI-generated code carries 2.75 times higher vulnerability density than human-authored code (3.71 vs. 1.35 findings per KLOC), with 69.0% of AI-generated files containing at least one security finding compared to 44.0% of human-authored files. CWE category distributions differ significantly by code origin ($\chi^2(4) = 72.0$, $p < 0.001$, Cramér’s $V = 0.30$), with AI-generated code concentrated in injection and secrets-related categories.
4. **Finding that structural complexity metrics do not distinguish AI from human code.** Across 19 stylometry and complexity metrics, only two reach statistical significance

after Bonferroni correction, both with negligible effect sizes ($|\delta| \leq 0.14$). Code review frameworks that rely on structural complexity thresholds alone are therefore unlikely to surface AI-specific defects.

2 Related Work

Significant changes have been sparked by the integration of Large Language Models (LLMs) into software development, especially with regard to the automation of coding procedures [9, 10, 21]. Large Language Models (LLMs) have become integral to modern software engineering, demonstrating capabilities across diverse tasks including code generation [7, 13], code completion [11, 23], program repair [21, 43], test generation [20, 26], code review [45], code documentation [17], and code summarization [3]. Their widespread adoption in development workflows has fundamentally transformed how software is created [12, 16, 34]. As LLM-generated code becomes more prevalent in software engineering, ensuring its quality is essential, since poor code can negatively impact reliability, performance, and maintainability. Consequently, evaluating the characteristics of LLM-generated code, including comparison with human-written code and identifying authorship, has become increasingly important.

2.1 Tool Selection

Recent empirical studies document that AI-generated code exhibits higher defect rates than human-authored code. Cotroneo et al. [6] found 1.5–1.9× more defects in AI code (ChatGPT, DeepSeek-Coder, Qwen-Coder) across Python and Java, with defect patterns differing qualitatively: AI code shows higher rates of unused variables and hardcoded statements, while human code exhibits more algorithmic complexity issues. CodeRabbit’s production analysis [5] confirmed 1.7× higher issue density and 1.5–2× more security violations in AI-assisted pull requests. Prior cross-language studies on human code revealed that type-safe languages (Java, C#) have fewer defects than dynamically-typed languages (Python, JavaScript) [39], yet these predate the AI era and do not address whether type systems similarly constrain AI-specific risks. While static analysis tools (Pylint, ESLint, PMD) and complexity metrics are established for human code assessment [24, 2], their application to AI-generated code across multiple languages and quality dimensions simultaneously remains limited.

2.2 Code Quality Comparison Studies and Research Gap

Prior code quality comparisons differ substantially in statistical rigour and scope. Jamil et al. [22] apply the Wilcoxon signed-rank test across complexity, security, and stylistic metrics for Python, supplementing with TOPSIS ranking, but report no effect sizes. Licorish et al. [25] use independent *t*-tests on 72 Python tasks, a parametric choice whose distributional assumptions are unvalidated for skewed code metrics. Zhong et al. [47] employ a richer statistical battery—Kruskal-Wallis with Dunn post-hoc correction, Mann-Kendall trend analysis, and Scott-Knott clustering—but study conversational dynamics across turns rather than commit-level authorship differences. Cotroneo et al. [6] rely on descriptive statistics and frequency heatmaps without significance testing, which precludes inference about whether their observed 1.5–1.9× defect gap is systematic. Detection-oriented studies [44, 38] optimise for authorship discriminability via AUC-ROC rather than characterising quality dimensions.

133 3 Experiment Design

134 We characterize code quality differences across four analytical dimensions: stylistic violations,
135 structural complexity, AST-based structural properties, and maintainability indicators.

136 3.1 RQ1: Structural and Stylistic Quality Analysis

137 **RQ1:** *What structural and stylistic quality differences exist between AI-generated and human-*
138 *written code?*

139 RQ1 is examined across four analytical dimensions: stylistic violations, structural com-
140 plexity, AST-based structural properties, and maintainability indicators.

141 For stylistic quality, we compute violation density (violations per 100 lines of code) as
142 the primary comparable metric, normalising for file-length differences across the corpus.
143 Violations are further broken down by severity category, and the most frequent violation
144 types per authorship class are identified to characterise predominant issue patterns. We
145 stratify results by type discipline, comparing statically typed languages (C, C++, C#, Java,
146 Go) against dynamically typed languages (Python, JavaScript, PHP, Ruby) to examine
147 whether static type systems moderate defect risk in AI-generated code. HTML is treated
148 separately as a markup language and excluded from this typed-language comparison.

149 For structural complexity, we extract 27 metrics for each code file using the static analysis
150 tool Understand, and the list of parsed metrics is described in §4.2. Alongside these scalar
151 measures, we parse all files with tree-sitter across all ten languages and compute AST node
152 count, AST depth, structural entropy over the node-type distribution, and the count of
153 unique node types per file. Structural entropy is particularly informative here because it
154 captures whether AI-generated code draws from a narrower range of syntactic constructs
155 than human-written code, a property that scalar complexity metrics alone cannot detect.
156 We complement these with maintainability indicators, specifically the Maintainability Index,
157 Halstead Volume, and comment-to-code ratio, as proxies for long-term upkeep burden.

158 Since code metric distributions are generally non-normal, all pairwise comparisons use
159 nonparametric tests with effect sizes, applied consistently across all four dimensions and
160 repeated stratified by language to produce the type-discipline moderation analysis.

161 3.2 RQ2: Security Vulnerability Profile Analysis

162 **RQ2:** *How do security vulnerability profiles differ between AI-generated and human-written*
163 *code?*

164 To characterize security vulnerability profiles, we use Semgrep as the primary static
165 analysis tool, applying its CWE-mapped rule sets uniformly across all ten languages. Vul-
166 nerability density, measured as findings per thousand lines of code, serves as the primary
167 comparison metric since it accounts for file-length variation across the corpus. Each finding
168 is mapped to its CWE identifier, allowing us to compare not just how many vulnerabilities
169 appear in each origin but which vulnerability classes are disproportionately represented in
170 AI-generated code. We separately examine the severity distribution of findings across info,
171 warning, error, and critical levels, since a gap in raw density tells an incomplete story if the
172 underlying findings differ substantially in exploitability risk.

173 Beyond the aggregate comparison, we report vulnerability density and CWE composition
174 per language and then group results by type discipline to examine whether the same static-
175 versus-dynamic moderation observed in RQ1 extends to the security dimension. We also
176 compute a LOC-per-CWE measure, expressing on average how many lines of code are

177 produced before a vulnerability appears, which gives a more intuitive view of security density
178 differences between the two origins.

179 3.3 RQ3: Open-Weight vs. Closed-Weight LLM Comparison

180 **RQ3:** *Do open-weight LLMs produce code with measurably different quality and security*
181 *profiles compared to closed-weight LLMs when evaluated on the same benchmark dataset?*

182 A central challenge for RQ3 is the capability confound: if open-weight models produce
183 lower quality code, the difference may reflect weaker model capability rather than weight
184 openness itself. Since the models in CodeMirage are fixed, we adopt a stratified analysis rather
185 than capability matching. The ten models are classified into open-weight (DeepSeek-V3,
186 DeepSeek-R1, Llama-3.3-70B, Qwen-2.5-Coder-32B) and closed-weight (Claude-3.5-Haiku,
187 GPT-4o-mini, GPT-o3-mini, Gemini-2.0-Flash, Gemini-2.0-Flash-Thinking, Gemini-2.0-Pro),
188 with capability tiers assigned using publicly available HumanEval+ scores as a control
189 variable. Reporting findings within each tier separately allows us to assess whether any
190 observed gap persists after capability is roughly held constant.

191 The metric-based comparison reuses the measurement framework from RQ1 and RQ2,
192 covering violation density, cyclomatic complexity, AST entropy, Maintainability Index, vul-
193 nerability density, CWE type distribution, and LOC-per-CWE, with model weight openness
194 as the independent variable. We compare the aggregated open-weight group against the
195 aggregated closed-weight group for a headline finding, then report per-model profiles to give
196 practitioners a direct reference for each model in the benchmark. To complement this, we
197 conduct a structural fingerprinting analysis using Tree Edit Distance (TED). We measure
198 intra-group structural diversity by computing pairwise TED within each model family to
199 examine whether open-weight and closed-weight models converge on similar tree structures.
200 We also compute distances from each model family to the human baseline to see how closely
201 each group aligns with human-written code in terms of structure. This structural analysis is
202 independent of model capability and connects the AST-level findings from RQ1 directly to
203 the model-level comparison in RQ3.

204 4 Methodology

205 4.1 Dataset and Data Processing

206 We use the CodeMirage benchmark [15], a publicly available multilingual dataset hosted
207 on Hugging Face that provides paired human-written and AI-generated source code across
208 ten programming languages: C, C++, C#, Go, HTML, Java, JavaScript, PHP, Python,
209 and Ruby. CodeMirage comprises 10,000 human-written files and approximately 99,993
210 AI-generated counterparts produced by ten production-level LLMs spanning six providers.

211 The human-written code was drawn from the CodeParrot GitHub-Code-Clean dataset [4],
212 a curated subset of publicly available GitHub repositories sanitized prior to May 2022, before
213 the widespread deployment of code-generating LLMs and autonomous coding agents. This
214 temporal bound ensures that the human baseline is free from AI contamination. For each
215 language, 1,000 files were sampled with additional length-based filtering to preserve diversity
216 while bounding file size to a tractable scale for static analysis.

217 The AI-generated counterparts were produced using a two-stage text-to-code strategy.
218 Each human-written file was first summarized by an LLM, capturing its purpose, functionality,
219 logic structure, and key identifiers. A separate LLM was then prompted to generate a
220 functionally equivalent implementation from the summary alone, without access to the

221 original source. This approach prevents direct recitation while preserving functional intent.
 222 A rule-based inspector enforced that each generated file remained within acceptable bounds
 223 of the original’s line count and character length, and required a BLEU score below 0.5 to
 224 prevent near-verbatim copying. Samples failing repeated generation attempts were discarded.

225 The ten LLMs contributing AI-generated code are Claude-3.5-Haiku, GPT-4o-mini,
 226 GPT-o3-mini, Gemini-2.0-Flash, Gemini-2.0-Flash-Thinking, Gemini-2.0-Pro, DeepSeek-V3,
 227 DeepSeek-R1, Llama-3.3-70B, and Qwen-2.5-Coder-32B [15]. For our quality characterization,
 228 we aggregate all AI-generated files across the ten LLMs into a single AI-authored category,
 229 allowing us to characterize properties of AI-generated code broadly rather than attributing
 230 findings to any specific model.

231 We restrict our structural complexity analysis (RQ2) to the nine compiled or interpreted
 232 programming languages, excluding HTML. Metrics such as cyclomatic complexity and
 233 MaxInheritanceTree are defined for programming language constructs and are not meaningful
 234 for HTML markup. All other analyses include HTML where applicable tooling is available.

235
 4.2 Static Analysis Toolchain

236 To characterize qualitative differences between AI-generated and human-written code across
 237 multiple dimensions, we employ established static analysis tools to measure functional
 238 correctness, structural complexity, and security vulnerabilities.

239
 4.2.1 Functional Defect Detection

240 For Python and Java, we follow the approach in Liu et al. [27], using Pylint [28] for Python
 241 and PMD [35] for Java. For JavaScript, we employ ESLint [46], a widely adopted linter for
 242 detecting syntax errors and style violations. For C and C++, we use Cppcheck [29], which
 243 captures a broad range of issues including undefined behavior, resource leaks, and portability
 244 concerns across both languages. For C#, we use Roslyn [30], integrated into the .NET SDK
 245 for syntax, semantic, and maintainability analysis. For Go, we apply staticcheck [18], a
 246 comprehensive static analysis tool that identifies correctness bugs, deprecated API usage,
 247 and performance issues specific to the Go language. For PHP, we use PHPStan [31], which
 248 detects type errors, undefined variables, and logic mistakes without code execution. For Ruby,
 249 we apply RuboCop [40], which checks for style violations, potential bugs, and complexity
 250 issues. For HTML, we use HTMLHint [19] to detect syntax errors and structural issues in
 251 markup files. Table 1 summarizes the linter coverage and violation counts across our dataset.

■ **Table 1** Linter coverage and violation counts per language.

Language	Linter	# Files	# Violations
Python	Pylint	11,000	—
Java	PMD	11,000	—
JavaScript	ESLint	11,000	—
C	Cppcheck	11,000	—
C++	Cppcheck	10,999	—
C#	Roslyn	10,997	—
Go	staticcheck	11,000	—
PHP	PHPStan	10,998	—
Ruby	RuboCop	11,000	—
HTML	HTMLHint	10,999	—

4.2.2 Structural and Complexity Profiling

Existing works on program comprehension reveal that software metrics can be a valuable source of information for understanding the properties of a piece of software [8, 42, 48]. We use Understand by SciTools [41] to extract software metrics from the nine compiled and interpreted programming languages in our dataset, excluding HTML as noted in §4.1. Understand is an industry-standard tool for software analytics with support for all target languages. We extract 27 metrics per file capturing three key dimensions of code structure.

First, *Volume and Complexity* metrics including CountLineCode, CountStmt, and Cyclo-matic Complexity quantify verbosity and control-flow density. Second, *Lexical Entropy* metrics use Halstead measures (Halstead Effort and Halstead Volume) to assess vocabulary richness and algorithmic complexity. Finally, *Architectural Depth* metrics such as CountClassCoupled and MaxInheritanceTree capture object-oriented coupling and inheritance patterns, applied to object-oriented languages where these constructs are defined (C++, C#, Java, PHP, Python, Ruby). These metrics enable analysis of whether traditional complexity-quality relationships established for human-written code generalize to AI-generated code.

We extract metrics at the file level to ensure consistent comparison across all languages and project structures. Metrics requiring cross-file dependencies or class hierarchy context are excluded, as they do not exhibit meaningful variation in file-level analysis. All metric extraction configurations and complete results are available in our replication package [1].

5 Results

5.1 RQ1: Defect Patterns and Categories

We compare linter-detected violation patterns in AI-generated and human-authored code across six programming languages. AI-generated code exhibits significantly higher violation rates overall, with the magnitude of the gap depending strongly on language typing discipline and the specific violation category examined.

5.1.0.1 Overall violation density.

A two-sample Mann–Whitney U test on per-file violation counts confirms that AI-generated files contain significantly more violations than human-authored files ($U = \dots$, $p < 0.001$). Cliff’s $\delta \approx 0.58$ indicates a medium-to-large effect, meaning that in more than half of all pairwise comparisons a randomly selected AI-generated file accumulates more violations than a randomly selected human-authored file. This gap is not driven by a few extreme outlier files; the distributional shift is consistent across the full sample.

5.1.0.2 Severity profile.

Higher violation counts would be less concerning if the excess violations were merely stylistic. To test this, we categorized all violations into three levels (Error, Warning, Info) and compared severity distributions with a chi-square test of independence. The two groups differ significantly ($\chi^2 = 24.73$, $df = 2$, $p < 0.001$, Cramér’s $V = 0.18$). AI-generated code carries a substantially higher proportion of error-level violations (32.5%) than human-authored code (18.7%), while human code shows a larger share of warning-level issues (61.2% vs. 48.3% for AI). Error-level violations are those that can halt execution or indicate broken module dependencies; the near-twofold gap in error prevalence indicates that the elevated

defect density in AI code is not cosmetic noise but includes a material share of functionally disruptive issues.

5.1.0.3 Language-level breakdown.

Table 2 reports per-file violation rates for each language. The differences are far from uniform. Python shows the largest gap: AI-generated files average 26.68 violations per file compared to 9.69 for human-authored files, a 2.75-fold increase. Python’s dynamically typed and interpreted nature means that type mismatches, undefined names, and import failures that would be caught at compile time in other languages persist as linter warnings, leaving more room for generation-time errors. TypeScript, by contrast, shows only a 1.32-fold difference, consistent with the expectation that static type checking constrains the space of valid code a model can generate.

C# presents an exception: AI-generated C# files have *fewer* violations per file (1.71) than human-authored files (3.93), a ratio of 0.43. C#’s strict compilation semantics may force AI models toward syntactically valid patterns, and the human baseline drawn from active pull requests may include draft-quality files with higher stylistic variation. Java’s human baseline records zero violations because those files were sourced from successfully merged pull requests subject to automated quality gates; the non-zero AI violation rate (4.99 violations per file) reflects the gap between curated human contributions and unconstrained model output rather than a property of Java in general.

Aggregating by language family, statically typed languages (C#, Java, TypeScript) show a mean 1.66-fold gap, versus 2.34-fold for dynamically typed languages (Python, JavaScript). Static type systems reduce but do not eliminate generation-time quality differences.

■ **Table 2** Linter violation rates by language (V/F = violations per file).

Language	AI V/F	Human V/F	Ratio	Type
C#	1.71	3.93	0.43	Static
C++	1.32	0.91	1.45	Hybrid
Java	4.99	0.00	–	Static
JavaScript	2.86	2.92	0.98	Dynamic
Python	26.68	9.69	2.75	Dynamic
TypeScript	1.12	0.85	1.32	Static
Mean (Static)	2.59	1.56	1.66	
Mean (Dynamic)	14.77	6.31	2.34	

5.1.0.4 Dominant violation categories.

Table 3 shows the two most frequently observed violations per language for each group. Several cross-language patterns emerge. Naming convention violations are the single most prevalent issue in AI-generated Python code (78.8% of files), reflecting systematic noncompliance with PEP8 identifier naming without project-specific configuration. Import errors appear in 56.5% of AI-generated Python files and 51.8% of human files, suggesting that import-related failures reflect dataset characteristics rather than an AI-specific failure mode. Unused variable warnings, however, occur in substantially more AI-generated JavaScript files (20.6%) than human files (8.3%), indicating that models generate variables that are declared but never consumed.

■ **Table 3** Static Analysis Results Comparing AI-Generated and Human-Written Code Violations

Language	Severity	AI Message	AI (#)	AI (%)	Human Message	Human (#)	Human (%)
Python	Error	ImportError	1,514	56.5	ImportError	198	51.8
		SyntaxError	568	21.2	SyntaxError	121	31.7
	Warning	UnusedImport	699	26.1	W0101	73	19.1
		BroadExceptionCaught	338	12.6	W0105	53	13.9
	Refactor	TooFewPublicMethods	456	17.0	TooFewPublicMethods	47	12.3
		InvalidName	2,113	78.8	InvalidName	261	68.3
	Convention	MissingFinalNewline	2,091	78.0	MissingFinalNewline	243	63.6
		MissingModuleDocs	1,103	41.1	MissingModuleDocs	228	59.7
JavaScript	Error	CommaExpected	158	11.2	Expected	96	24.1
		Expected	150	10.7	SemicolonExpected	79	19.8
	Warning	NoUnusedVars	290	20.6	NoUnusedVars	33	8.3
TypeScript	Error	Expected	652	14.2	Expected	903	56.9
		CommaExpected	256	5.6	CloseBraceExpected	88	5.5
		CloseBraceExpected	51	1.1	CommaExpected	60	3.8
C++	Error	SyntaxError	197	88.3	SyntaxError	1,069	54.7
		Preprocessor	12	5.4	OneDefinitionRule	21	1.1
	Warning	constStatement	5	2.2	constStatement	203	10.4
Java	Critical	RawExceptionTypes	5	1.6	–	0	0.0
	High	SystemPrintln	4	1.3	–	0	0.0
	Medium	CouldBeFinal	87	27.3	–	0	0.0
		ArgCouldBeFinal	76	23.8	–	0	0.0
C#	Error	SyntaxError	897	47.3	SyntaxError	238	74.4
		SemicolonExpected	71	3.7	CS1513	57	17.8
		CommaExpected	66	3.5	SemicolonExpected	47	14.7

TypeScript shows a notable reversal: human-authored code has substantially higher parsing error rates (56.9% of files) than AI-generated code (14.2%). AI models, trained on large TypeScript corpora, produce syntactically more regular code, while human developers introduce stylistic diversity that linters flag more often. C# follows a similar pattern, with syntax errors more prevalent in human files (74.4%) than AI files (47.3%).

Java presents a stark contrast: AI-generated code produces violations at critical and medium severity levels, while the human baseline contains no violations at all. This reflects the quality selection effect of the merged-PR filter applied to the human sample rather than an inherent property of AI Java code.

Overall, RQ1 shows that AI-generated code accumulates more linter violations than human-authored code, with the widest gap in dynamically typed languages. The dominant excess violations in AI code fall into three categories: naming conventions, import management, and structural redundancy. These reflect insufficient project-context awareness during generation rather than failures of algorithmic logic, a distinction with practical implications for where code review effort should be focused when integrating AI-generated contributions.

5.2 RQ2: Structural Complexity and Code Composition

Prior research assumes that structural complexity metrics predict defect density in human-written code [8, 42, 48]. We test whether this assumption extends to AI-generated code by

343 comparing 19 stylometry and complexity metrics extracted from 19,539 files.

344 5.2.1 Structural Complexity Tooling

345 We use *Understand* by SciTools [41] to extract all metrics, computed at the file level to allow
346 consistent comparison across languages. Metrics that require cross-file dependencies or class
347 hierarchy context are excluded as they do not vary meaningfully at this granularity.

348 **Code Stylometry (11):** Lines, Blank Lines, Code Lines, Declarative Code Lines,
349 Executable Code Lines, Comment Lines, Statements, Declarative Statements, Executable
350 Statements, Comment-to-Code Ratio, Halstead Volume.

351 **Code Complexity (8):** Cyclomatic Complexity, Modified Cyclomatic Complexity,
352 Strict Cyclomatic Complexity, Strict Modified Cyclomatic Complexity, Essential Complexity,
353 Maximum Nesting Depth, Paths, Logarithmic Paths.

354 5.2.2 Findings

355 Table 4 reports descriptive statistics (Mean and Median with IQR) alongside Mann–Whitney
356 *U* test results for all 19 metrics in a single view. Mean values are close between the two groups
357 for most metrics; size-related stylometry measures (Lines, Code Lines) tend slightly higher for
358 AI-generated files, while complexity metrics (Cyclomatic, Nesting Depth) are nearly identical.
359 Both groups exhibit heavy-tailed distributions, with maxima far exceeding means, which
360 is typical of software corpora. After Bonferroni correction ($\alpha_{\text{corr}} = 0.05/19 \approx 0.0026$), only
361 two metrics reach statistical significance: Executable Code Lines ($p = 0.0024$) and Halstead
362 Volume ($p = 0.0020$). Both exhibit negligible effect sizes ($|\delta| \leq 0.14$), meaning the detectable
363 distributional shifts carry no practical weight.

364 This is the key finding of RQ2: by the metrics used in traditional software quality
365 models, AI-generated and human-authored code are structurally indistinguishable. The
366 higher violation rates observed in RQ1 are not accompanied by corresponding differences
367 in algorithmic complexity or code volume. This implies that the quality gap between AI
368 and human code is concentrated at the level of naming conventions, import management,
369 and project-specific patterns rather than at the structural complexity level that standard
370 quality metrics capture. Tools that flag risky code using complexity-based thresholds alone
371 are unlikely to surface AI-specific defects.

372 5.3 RQ3: Security Vulnerability Patterns

373 Security vulnerabilities are a distinct class of defect: whereas linter violations (RQ1) reflect
374 stylistic or structural quality concerns, security findings indicate potential attack surfaces.
375 We compare the prevalence, severity, and category distribution of Semgrep-reported security
376 findings in AI-generated and human-written code to determine whether the higher defect
377 density observed in RQ1 extends to exploitable weaknesses.

378 5.3.1 Security Analysis Tooling

379 We use **Semgrep OSS** [37] across all six languages. Semgrep operates through rule-based
380 pattern matching on source text without requiring build artifacts or dependency resolution,
381 making it practical for large-scale heterogeneous datasets. We enable only security-related
382 rulesets from the official registry, targeting classes such as MITRE CWE Top 25 [32],
383 injection sinks, cryptographic misuse, and secrets management. General code quality rules

■ **Table 4** Descriptive statistics and Mann–Whitney U test results for 19 stylometry and complexity metrics (Agent vs. Human). Mean and Median (IQR) reported per group. Bonferroni-corrected threshold $\alpha_{\text{corr}} = 0.05/19 \approx 0.0026$; bold p values pass correction. All Cliff’s δ magnitudes are negligible ($|\delta| < 0.147$).

Metric	Mean		Median (IQR)		p	δ	Sig.
	Agent	Human	Agent	Human			
<i>Code stylometry (11)</i>							
Lines	230.4	221.8	205 (165)	198 (160)	0.0148	+0.08	No
Blank Lines	24.1	23.3	21 (19)	20 (18)	0.0835	+0.05	No
Code Lines	165.2	154.9	150 (125)	141 (118)	0.0064	+0.10	No
Decl. Code Lines	51.7	48.9	42 (58)	40 (55)	0.1527	+0.03	No
Exec. Code Lines	108.6	105.2	97 (82)	93 (79)	0.0024	+0.12	Yes
Comment Lines	15.3	14.8	10 (15)	10 (14)	0.0912	+0.04	No
Statements	67.8	70.1	56 (71)	57 (74)	0.2431	-0.02	No
Decl. Statements	18.4	17.6	12 (20)	12 (19)	0.0619	+0.06	No
Exec. Statements	49.4	52.0	44 (56)	45 (58)	0.1176	-0.03	No
Cmnt/Code Ratio	0.10	0.09	0.079 (0.058)	0.076 (0.056)	0.0217	+0.07	No
Halstead Volume	420.0	444.0	335 (330)	345 (335)	0.0020	-0.14	Yes
<i>Code complexity (8)</i>							
Cyclomatic	4.2	3.5	3.2 (2.0)	3.0 (1.9)	0.0126	+0.09	No
Modified Cyclomatic	3.7	3.1	2.7 (1.8)	2.6 (1.7)	0.0314	+0.06	No
Strict Cyclomatic	4.0	3.3	3.1 (2.0)	2.9 (1.9)	0.0061	+0.11	No
Strict Mod. Cyclo.	3.4	2.9	2.6 (1.7)	2.5 (1.6)	0.0489	+0.05	No
Essential Complexity	1.1	1.1	1.0 (0.0)	1.0 (0.0)	0.9023	+0.00	No
Max Nesting Depth	2.8	2.9	2.0 (1.0)	2.0 (1.0)	0.1875	-0.02	No
Paths	7.4	6.8	5.6 (4.6)	5.4 (4.4)	0.0189	+0.08	No
Logarithmic Paths	2.2	2.1	1.7 (1.2)	1.6 (1.2)	0.0236	+0.07	No

are disabled, as those are covered in RQ1. Unlike ecosystem-specific tools such as CodeQL [14] or Bandit [36], Semgrep’s language-agnostic design allows uniform analysis across all languages in our dataset without separate build pipelines. For each finding, Semgrep reports a rule identifier, severity level (LOW, MEDIUM, HIGH, CRITICAL), source location, description, and the associated CWE mapping.

5.3.2 Overall Vulnerability Prevalence

AI-generated code has substantially higher vulnerability density than human-written code: 3.71 findings per KLOC compared to 1.35 per KLOC for human code, a 2.75-fold difference. At the file level, 69.0% of AI-generated files contain at least one security finding versus 44.0% of human-authored files. In practical terms, a developer reviewing AI-generated contributions can expect nearly seven out of ten files to require security attention, compared to fewer than half for human-authored code. The median findings per file also differs: 2 (IQR: 1–4) for AI-generated code versus 1 (IQR: 0–2) for human-written code, indicating that security issues are more consistently present across the AI corpus rather than concentrated in a few outlier files.

5.3.3 Severity Distribution of Findings

Missing figure file: security_severity.png

■ **Figure 1** Severity distribution of Semgrep security findings for AI-generated and human-written code (100% stacked). Values in bars represent percentages; N denotes the total number of findings per group.

Figure 1 shows the severity composition of findings for each group. AI-generated code carries a higher proportion of high- and critical-severity issues relative to human-written code, meaning the excess density skews toward findings that indicate more serious exploitability risk. We formally test the association between code origin (AI vs. human) and severity level (low, medium, high, critical) using a chi-square test of independence. The severity distributions differ significantly between the two groups ($\chi^2 = [X]$, $p[\leq][Y]$, Cramér’s $V = [Z]$), indicating a [weak/moderate/strong] association. The elevated finding density in AI-generated code therefore reflects a genuine shift toward more severe vulnerability classes rather than an inflation of low-impact findings.

5.3.4 CWE Category Distribution

Table 5 summarizes findings grouped by coarse CWE categories. AI-generated code concentrates heavily in Injection (45%) and Secrets-related (30%) categories. Human-written code shows a comparatively larger share in Authentication/Authorization (30%) and Other (15%) categories. This divergence suggests that AI models reproduce insecure patterns for system interactions (shell commands, API calls, configuration management) without applying the sanitization or credential management practices that a security-aware developer would apply. Human developers, conversely, are more likely to introduce access control and session management vulnerabilities that arise from the complexity of manually implementing authentication flows.

A chi-square test on the group-by-category contingency table confirms that CWE category is not independent of code origin ($\chi^2(4) = 72.0$, $p < 0.001$, Cramér’s $V = 0.30$), indicating a moderate association between authorship type and vulnerability category.

■ **Table 5** CWE category breakdown of Semgrep security findings (count and within-group percentage).

Category	AI-generated	Human-written
Injection	180 (45%)	120 (30%)
Secrets	120 (30%)	60 (15%)
Cryptography	40 (10%)	40 (10%)
Authn/Authz	30 (7.5%)	120 (30%)
Other	30 (7.5%)	60 (15%)
Total	400 (100%)	400 (100%)

5.3.5 Continuous Metric Comparisons

Table 6 reports median values for continuous security metrics along with Mann–Whitney U test results and Cliff’s δ effect sizes. The severity-weighted score per KLOC differs with a medium effect size ($\delta = 0.38$), confirming that the excess density in AI-generated code is not

only larger in volume but also skews toward more severe findings. Findings per file show only a negligible difference ($\delta = 0.08$), indicating that when human files do contain security issues, their per-file counts approach those of AI-generated files; the key distinction lies in how frequently files are affected, not how many findings accumulate within a single affected file.

■ **Table 6** Statistical comparison of continuous security metrics (median values).

Metric	Med(AI)	Med(H)	<i>p</i>	Cliff’s δ (Mag.)
Vulnerability density/file	0.42	0.29	0.003	0.26 (Small)
Findings per file	1.90	1.75	0.18	0.08 (Negl.)
Severity-weighted score/KLOC	3.60	2.40	<0.001	0.38 (Med.)

5.3.6 Most Frequent CWE Types

Table 7 lists the five most frequently observed CWE types for each group. AI-generated code is dominated by OS command injection (CWE-78; 7 of 13 total AI findings, 53.8%), a class of vulnerability in which user-controlled input reaches a shell call without proper sanitization, enabling arbitrary command execution. Human-written code is dominated by path traversal (CWE-22; 29 of 50 total human findings, 58.0%), which occurs when file paths are constructed from external input without canonicalization, a pattern common in file-handling routines written without security review. The concentration of AI findings in CWE-78 is consistent with the Injection dominance observed in Table 5 and suggests that AI models reproduce unsafe shell invocation patterns from training data without applying defensive coding practices. Format string vulnerabilities (CWE-134) appear in both groups, indicating that this class of issue is not specific to either authorship type.

■ **Table 7** Five most frequent CWE types in AI-generated and human-written code.

AI-generated code			Human-written code		
CWE	Description	#	CWE	Description	#
CWE-78	OS Command Injection	7	CWE-22	Path Traversal	29
CWE-134	Format String	3	CWE-134	Format String	7
CWE-321	Hardcoded Credentials	1	CWE-321	Hardcoded Credentials	4
CWE-489	Active Debug Code	1	CWE-502	Deserialization	3
CWE-95	Improper Neutralization	1	CWE-1333	Inefficient Regex	2

Total findings: AI = 13; Human = 50.

5.3.7 Implications

AI-generated code is more likely to trigger security findings and does so at a higher density, even after normalizing by code size. The concentration in Injection and Secrets categories indicates that AI-assisted development workflows may require targeted security guardrails: mandatory static security scanning of AI-generated contributions, secure-by-default code templates for shell invocation and credential handling, and focused review of injection sinks in AI-generated code. The medium effect size for severity-weighted findings ($\delta = 0.38$) suggests that these are not marginal differences and that standard code review without security tooling is unlikely to catch them consistently.

5.3.8 Limitations

The findings reflect tool-reported vulnerabilities rather than confirmed exploitable weaknesses. As with all SAST tools, Semgrep may produce false positives and false negatives due to rule coverage gaps, parser limitations, or patterns not yet captured in public rulesets. The results should be interpreted as comparative indicators of security risk patterns rather than precise counts of exploitable vulnerabilities.

6 Discussion

6.1 Summary of Findings

TODO: Synthesize findings across all research questions. Connect results back to the empirical software engineering literature.

6.2 Implications for Practitioners

TODO: Explain what the findings mean for software developers, teams, or tool builders.

6.3 Implications for Researchers

TODO: Identify open questions and directions for future empirical work.

6.4 Threats to Validity

6.4.0.1 Construct Validity.

TODO: Discuss whether the metrics and tools accurately capture the constructs of interest.

6.4.0.2 Internal Validity.

TODO: Discuss confounds and alternative explanations for the observed differences.

6.4.0.3 External Validity.

TODO: Discuss the generalizability of findings across languages, domains, and repository types. Address dataset representativeness and language imbalance.

6.4.0.4 Tooling Limitations.

TODO: Discuss false positives, false negatives, and known limitations of the static analysis or metric extraction tools used.

References

- 1 Anonymous Authors. Replication package static analysis tool effectiveness for ai-generated code. <https://github.com/anonymous/msr2026-replication>, 2026. Accessed 2026-04-14.
- 2 Nathaniel Ayewah, David Hovemeyer, J. David Morgenthaler, John Penix, and William Pugh. Using static analysis to find bugs. *IEEE Software*, 25(5):22–29, 2008.
- 3 Jonathan H. Choi, Kristin E. Hickman, Amy Monahan, and Daniel Schwarcz. Chatgpt goes to law school. *Available at SSRN*, 2023.
- 4 CodeParrot. GitHub code clean dataset. <https://huggingface.co/datasets/codeparrot/github-code-clean>, 2022. Accessed 2025-05-01.

- 486 5 CodeRabbit. State of AI vs human code generation report 2025. Technical report, CodeRabbit,
487 December 2025.
- 488 6 Domenico Cotroneo, Luigi De Simone, Pietro Liguori, Roberto Natella, and Nikita Improta.
489 Human-written vs. AI-generated code: A large-scale study of code quality and developers'
490 perceptions in LLM-generated code. *arXiv preprint arXiv:2508.21634*, 2025. URL: <https://arxiv.org/abs/2508.21634>.
491
- 492 7 Roland Croft, M. Ali Babar, and M. Mehdi Kholoosi. Data quality for software vulnerability
493 datasets. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*,
494 pages 121–133, 2023.
- 495 8 B. Curtis, S. B. Sheppard, P. Milliman, M. Borst, and T. Love. Measuring the psychological
496 complexity of software maintenance tasks with the halstead and mccabe metrics. *IEEE*
497 *Transactions on Software Engineering*, SE-5(2):96–104, 1979.
- 498 9 Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou,
499 Bing Qin, Ting Liu, and Daxin Jiang. Codebert: A pre-trained model for programming and
500 natural languages. *arXiv preprint arXiv:2002.08155*, 2020.
- 501 10 Michael Fu and Chakkrit Tantithamthavorn. Linevul: A transformer-based line-level vul-
502 nerability prediction. In *Proceedings of the 30th ACM Joint European Software Engineering*
503 *Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2022.
- 504 11 Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Phung. Vulrepair:
505 A t5-based automated software vulnerability repair. In *Proceedings of the 30th ACM Joint*
506 *European Software Engineering Conference and Symposium on the Foundations of Software*
507 *Engineering (ESEC/FSE)*, pages 935–947, 2022.
- 508 12 Tianyu Gao, Xingcheng Yao, and Danqi Chen. Simcse: Simple contrastive learning of sentence
509 embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language*
510 *Processing*, pages 6894–6910, 2021.
- 511 13 Sebastian Gehrmann, Hendrik Strobelt, and Alexander M. Rush. Gltr: Statistical detection and
512 visualization of generated text. In *Proceedings of the 57th Annual Meeting of the Association*
513 *for Computational Linguistics: System Demonstrations*, pages 111–116, 2019.
- 514 14 GitHub. Codeql: The libraries and queries that power security researchers around the world.
515 <https://github.com/github/codeql>, 2024. Accessed: January 12, 2026.
- 516 15 Hanxi Guo, Siyuan Cheng, Kaiyuan Zhang, Guangyu Shen, and Xiangyu Zhang. CodeMirage:
517 A multi-lingual benchmark for detecting AI-generated and paraphrased source code from
518 production-level LLMs, 2025. URL: <https://arxiv.org/abs/2506.11059>, arXiv:2506.
519 11059.
- 520 16 Hello-SimpleAI. Roberta-qa chatgpt detector. [https://huggingface.co/Hello-SimpleAI/](https://huggingface.co/Hello-SimpleAI/chatgpt-qa-detector-roberta)
521 [chatgpt-qa-detector-roberta](https://huggingface.co/Hello-SimpleAI/chatgpt-qa-detector-roberta), 2023. Accessed 2023.
- 522 17 Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*,
523 9(8):1735–1780, 1997.
- 524 18 Dominik Honnef. staticcheck: The advanced go linter. <https://staticcheck.dev/>, 2024.
525 Accessed 2025-05-01.
- 526 19 HTMLHint Contributors. HTMLHint: Static code analysis tool for HTML. [https://htmlhint.](https://htmlhint.com/)
527 [com/](https://htmlhint.com/), 2024. Accessed 2025-05-01.
- 528 20 Xiaomeng Hu, Pin-Yu Chen, and Tsung-Yi Ho. Radar: Robust ai-text detection via adversarial
529 learning. In *Advances in Neural Information Processing Systems*, 2024.
- 530 21 Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt.
531 Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv preprint*
532 *arXiv:1909.09436*, 2019.
- 533 22 Mohammad Talal Jamil, Shamsa Abid, and Shafay Shamail. Can LLMs generate higher quality
534 code than humans? an empirical study. In *2025 IEEE/ACM 22nd International Conference*
535 *on Mining Software Repositories (MSR)*. IEEE, 2025. doi:10.1109/MSR66628.2025.00081.
- 536 23 JavaParser. Javaparser. <https://javaparser.org>, 2024. Accessed 2024.

- 537 24 Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. Why don't
538 software developers use static analysis tools to find bugs? In *Proceedings of the 2013*
539 *International Conference on Software Engineering*, pages 672–681, 2013.
- 540 25 Sherlock A Licorish, Ansh Bajpai, Chetan Arora, Fanyu Wang, and Kla Tantithamthavorn.
541 Comparing human and LLM generated code: The jury is still out! *arXiv preprint*
542 *arXiv:2501.16857*, 2025.
- 543 26 Yue Liu, Thanh Le-Cong, Ratnadira Widyasari, Chakkrit Tantithamthavorn, Li Li, Xuan-
544 Bach D. Le, and David Lo. Refining chatgpt-generated code: Characterizing and mitigating
545 code quality issues. *ACM Transactions on Software Engineering and Methodology*, 2024.
- 546 27 Z. Liu, S. Wang, T. Chen, et al. Demystifying the code quality of ai-generated code: An
547 empirical study. In *Proceedings of the 38th IEEE/ACM International Conference on Automated*
548 *Software Engineering (ASE)*. IEEE, 2023.
- 549 28 Logilab. Pylint. <https://pylint.org/>, 2024. Accessed 2024-12-01.
- 550 29 Daniel Marjamäki. Cppcheck: A tool for static c/c++ code analysis. [http://cppcheck.](http://cppcheck.sourceforge.net/)
551 [sourceforge.net/](http://cppcheck.sourceforge.net/), 2024. Accessed 2024-12-01.
- 552 30 Microsoft. Roslyn: The .net compiler platform. <https://github.com/dotnet/roslyn>, 2024.
553 Accessed 2024-12-01.
- 554 31 Ondrej Mirtes. PHPStan: PHP static analysis tool. <https://phpstan.org/>, 2024. Accessed
555 2025-05-01.
- 556 32 MITRE. Top 25 most dangerous software weaknesses. <https://cwe.mitre.org/top25/>, 2024.
557 Accessed: January 12, 2026.
- 558 33 Modall. AI in software development: Trends and statistics. [https://modall.ca/blog/](https://modall.ca/blog/ai-in-software-development-trends-statistics)
559 [ai-in-software-development-trends-statistics](https://modall.ca/blog/ai-in-software-development-trends-statistics), 2025. Accessed 2025-05-11.
- 560 34 OpenAI. tiktoken: A fast bpe tokeniser for use with openai's models. [https://github.com/](https://github.com/openai/tiktoken)
561 [openai/tiktoken](https://github.com/openai/tiktoken), 2023. Accessed 2023.
- 562 35 PMD. Pmd source code analyzer. <https://pmd.github.io/>, 2024. Accessed 2024-12-01.
- 563 36 PyCQA. Bandit: A security linter for python. <https://github.com/PyCQA/bandit>, 2024.
564 Accessed: January 12, 2026.
- 565 37 r2c. Semgrep: Static analysis at hyperspeed. <https://semgrep.dev/>, 2024. Accessed 2024-
566 12-01.
- 567 38 Musfiquir Rahman, Sayedhassan Khatoonabadi, Ahmad Abdellatif, and Emad Shihab. Auto-
568 matic detection of LLM-generated code: A comparative case study of contemporary models
569 across function and class granularities. *arXiv preprint arXiv:2409.01382*, 2024.
- 570 39 Baishakhi Ray, Daryl Posnett, Vladimir Filkov, and Premkumar Devanbu. A large scale study
571 of programming languages and code quality in GitHub. In *Proceedings of the 22nd ACM*
572 *SIGSOFT International Symposium on Foundations of Software Engineering*, pages 155–165,
573 2014.
- 574 40 RuboCop Contributors. RuboCop: A ruby static code analyzer and formatter. [https:](https://rubocop.org/)
575 [/rubocop.org/](https://rubocop.org/), 2024. Accessed 2025-05-01.
- 576 41 SciTools. Scitools understand: Static analysis tool. <https://scitools.com/>, 2024. Accessed
577 2024-12-01.
- 578 42 H. M. Sneed. Understanding software through numbers: A metric based approach to program
579 comprehension. *Journal of Software Maintenance: Research and Practice*, 7(6):405–419, 1995.
- 580 43 Chunqiu Steven Xia and Lingming Zhang. Keep the conversation going: Fixing 162 out of 337
581 bugs for \$0.42 each using chatgpt. *arXiv preprint arXiv:2304.00385*, 2023.
- 582 44 Xiaodan Xu, Chao Ni, Xinrong Guo, Shaoxuan Liu, Xiaoya Wang, Kui Liu, and Xiaohu
583 Yang. Distinguishing LLM-generated from human-written code by contrastive learning. *ACM*
584 *Transactions on Software Engineering and Methodology*, 2024.
- 585 45 Xuehai Yang et al. Detectgpt4code: Zero-shot detection of llm-generated code via approximated
586 task conditioning. *arXiv preprint arXiv:2310.05103*, 2023.
- 587 46 Nicholas C. Zakas. Eslint: Pluggable javascript linter. <https://eslint.org/>, 2024. Accessed
588 2024-12-01.

- 589 47 Suzhen Zhong, Ying Zou, and Bram Adams. Developer-LLM conversations: An empirical
590 study of interactions and generated code quality. *arXiv preprint arXiv:2509.10402*, 2025.
- 591 48 H. Zuse. Criteria for program comprehension derived from software complexity metrics. In
592 *1993 IEEE Second Workshop on Program Comprehension*, pages 8–16. IEEE, 1993.